Scaffold Workflow Guide

A neutral, copy-and-adapt workflow for AI-assisted development with Claude Code and Google Antigravity: how each part is generated and configured, how its rules work, and what it is for.

The workflow came out of a long project and was stripped of everything specific to it. What is left is a set of files, rules and small scripts you can copy into any repository. Replace the placeholders `<PROJECT>`, `<area>`, `<build-cmd>` and `<test-cmd>`, keep the structure.

How to read this guide. Every section has the same shape: a **TL;DR** box, the detailed explanation, and a closing block of **Practical examples**. The examples use a fictional web app, Taskly (a Python server, a web client and a mobile client); the captured outputs come from the real scripts run on `examples/demo`.

Companions. The one-page summary with links to every file is `docs/WORKFLOW.md`. Everything checkable is checked by `python scripts/selftest.py`. This PDF is rebuilt with `python scripts/build_workflow_pdf.py`.

Contents

1. Why a workflow . . . 3
2. Repository layout . . . 5
3. CLAUDE.md and targeted retrieval . . . 7
4. The docs system . . . 9
5. Subagent roles and routing . . . 11
6. Readiness Gate . . . 13
7. Evidence-based escalation . . . 15
8. Necessity score, autonomy threshold, ledger and agent_audit . . . 16
9. Consistency gate: spec files, drift, divergences . . . 18
10. Docs audit . . . 20
11. CI audit . . . 21
12. Multi-operator sync . . . 22
13. Writing rules as positive instructions . . . 24
14. Bootstrapping step by step . . . 25
15. Maintenance and metrics . . . 27

1. Why a workflow

TL;DR AI agents fail in predictable ways: they reread everything, guess when a task is vague, spend the expensive model on cheap work, let docs and platforms drift, and collide with each other. The workflow turns each failure into a rule, a file and a script that checks it.

The failures and the mechanism that answers each

Failure	Mechanism	Where it lives
Agents read whole files and burn tokens	Targeted retrieval: one index, grep then read a slice	`CLAUDE.md` section 1, `docs/progress.md`
A vague task produces a wrong diff	A planner turns it into Work Orders; the Readiness Gate refuses unready tasks	`.claude/agents/planner.md`, skill `work-order`
The expensive model does cheap work	Role routing, a necessity score with an autonomy threshold, a ledger	`CLAUDE.md` section 3, `docs/agents/ledger.txt`
Agents say DONE without proof	Evidence-based status: DONE needs a passing command	agent files, `CLAUDE.md` section 3
Docs go stale and hide rules	One sheet for each area with fixed `Open work` and `Rules` sections, audited by a score	`docs/progress/`, `scripts/docs_audit.py`
Platforms drift apart silently	A registry of what is promised and a gate that lists differences	`docs/spec/`, `scripts/consistency.py`
CI wastes minutes	Rules for timeouts, concurrency and ignored branches, audited	`scripts/ci_audit.py`
Parallel agents collide or go silent	A sync protocol: one writer for each file, inboxes, states, deliveries, reviews	`SYNC.md`, `sync/`, `tools/sync_*.py`
Rules are ignored or misread	Rules written as positive instructions, retired by deletion	`CLAUDE.md` section 2

The principle: separate deciding from executing

A top-tier model is expensive on output tokens and excellent at reasoning, so it produces short decisions and plans. A cheap model is good at writing a lot of code from a locked-down order. The router picks the agent by **how well the task is already specified**, not by how big it looks. A one-line request can be vague and dangerous; a ten-file rename can be mechanical and safe.

Everything is checkable

A rule that nobody can check decays. Each part has a script: `docs_audit.py` measures the docs, `consistency.py` compares facts across files, `agent_audit.py` reads the ledger, `ci_audit.py` reads the workflow, `sync_check.py` reads the sync state. `selftest.py` runs all of them on the demo project and also breaks the demo on purpose to prove each script notices.

Practical examples

Example 1: a vague request, with and without the workflow

Request: "make search better". Without the workflow a cheap coder starts editing: nine files touched, about 40k tokens, a result nobody can verify. With it, the request goes to `planner`, which returns three Work Orders (a filter parser, an index tweak, a UI chip), each with files, a test command and an agent. Two pass the Readiness Gate and go to `coder`; one fails question 5 (it needs a product decision) and becomes a line in `docs/open-decisions.md`.

Example 2: two agents, one file

Two operators both need the build file. In the sync protocol the file split in section 2 of `SYNC.md` names the owner of every file; the shared build file is allowed only "one line each, named in the Work Order". Neither agent overwrote the other, and the review diff against the named perimeter proved it.

Example 3: the cost nobody sees
The ledger of the demo shows 12 runs: the one `deep` run (88,000 tokens, 420 s) was logged with necessity 2, and `agent_audit.py` says so out loud: "expensive agent used 1 time(s) with necessity <= 3". Without the ledger the same spend is invisible.

2. Repository layout

TL;DR A handful of fixed places: agent notes at the root, subagent and skill definitions under `.claude/`, docs under `docs/`, audit scripts under `scripts/`, sync tools under `tools/`, protocol templates in `SYNC.md` and `sync/`, and a filled demo under `examples/demo/`. Scripts take `--root DIR`.

The tree

```
<repo>/
  CLAUDE.md  AGENTS.md  GEMINI.md        session notes for Claude Code, generic agents, Antigravity
  SYNC.md  sync/                         sync protocol TEMPLATES (the live copy lives on branch sync/<name>)
    _template.md  operators.toml  inbox/README.md
  .claude/agents/                        planner, coder, mechanic, light, deep, research, ops
  .claude/skills/                        work-order, docs-loop, docs-map
  docs/progress.md                       the one index
  docs/progress/<area>.md                one sheet for each area: Open work, Rules (do not break)
  docs/ROADMAP.md  docs/open-decisions.md  docs/WORKFLOW.md
  docs/agents/ledger.txt                 subagent runs, append only
  docs/spec/                             specs.toml config-keys.toml platforms.toml divergences.toml
                                         known.toml consistency.toml CONSISTENCY.md
  scripts/                               docs_audit consistency agent_audit ci_audit selftest
                                         build_workflow_pdf
  tools/                                 sync_hdr sync_check sync_watch
  .github/workflows/ci.yml  .ci-audit.toml
  examples/demo/                         "Taskly": a small filled project the scripts run on
```

What lives where

Place	Main branch	Sync branch
`CLAUDE.md`, `AGENTS.md`, `GEMINI.md`, `.claude/`, `docs/`, `scripts/`	yes	inherited from main
`SYNC.md`, `sync/` templates, `tools/sync_*.py` (templates)	yes: templates only	the live, filled copy
`sync/<ID>.md`, `sync/inbox/<ID>.md`, `sync/wo/<ID>.md`, Log rows	never	yes, only here

Conventions shared by the scripts

- Plain Python 3.11 or newer (`tomllib`), no dependencies. Only the PDF builder needs `reportlab`.
- `--root DIR` points a script at another project folder; the default is the folder above `scripts/` (or `tools/`). The sync tools also read the environment variable `SYNC_ROOT`.
- Exit code 0 means healthy, 1 means a finding, 2 means a usage error. CI can run any script and trust the code.
- Nothing writes to your docs except the tool you asked to write (`sync_hdr.py` writes one block; `docs_audit.py log` appends a row).

Practical examples

Example 1: adapting the layout to Taskly

Taskly keeps its server in `src/server/`, so `docs/progress/api.md` lists `src/server/app.py` in its main-code column, and the `[[match]]` block in `docs/spec/consistency.toml` points at the same file. Nothing else in the layout changed.

Example 2: one command proves the kit works

```
$ python scripts/selftest.py
selftest: 40/40 checks passed
```

Run it after copying the kit into a new repository: it compiles every script, runs each on the demo, and checks that deliberately broken copies of the demo are caught.

Example 3: why `examples/demo` exists
A template with placeholders is hard to judge. The demo is a complete small project with real numbers (a score of 717, two gate blockers, a ledger with a failed run): open it next to your own files to see what "filled in" looks like. It is also the fixture the audit scripts are tested on.

3. CLAUDE.md and targeted retrieval

TL;DR `CLAUDE.md` is the always-loaded briefing: how to find information, the rules of engagement, agent routing, invariants, task closure and the sync handshake. It stays small by pointing to `docs/progress.md`. Agents search with grep and read slices, never whole files.

Generating it

Copy `CLAUDE.md` from the scaffold and replace `<PROJECT>`, `<main-branch>`, `<build-cmd>` and `<test-cmd>`; delete the rows of the quick-lookup table that do not apply. `AGENTS.md` mirrors section 3 for tools that read that name; `GEMINI.md` holds only what differs for Antigravity (the model mapping and the branch-naming question). When they differ, `CLAUDE.md` wins.

The six sections

Section	What it holds	Why it is there
1 Targeted retrieval	A quick-lookup table of exact commands	An agent that knows the command does not explore
2 Rules of engagement	Branch naming, selective tests, secrets, licences, how to write rules	Behaviour that must be the same in every session
3 Subagents and routing	Roles, Readiness Gate, necessity score, ledger, escalation	Separate deciding from executing
4 Global invariants	Facts true across the whole project	Things no area sheet owns
5 Task closure	A six-step checklist run before declaring a task done	Docs and checks stay in step with the code
6 Multi-operator sync	Handshake, delivery, inbox, heartbeat, sign-off	Several agents can join with one sentence

Targeted retrieval, concretely

```
grep -n -A 15 '^## Open work' docs/progress/<area>.md      # what is left to do in an area
grep -n -A 20 '^## Rules'     docs/progress/<area>.md      # the binding rules of an area
grep -n -m5 '<symbol>' <file>                              # find code in a big file, then Read offset/limit
```

The reason is arithmetic. A 350 KB source file is roughly 90,000 tokens; the section an agent needs is usually 40 lines. Reading the slice costs about 1 percent. The index (`docs/progress.md`) exists so the agent never has to search for the search command.

Keep it lean

- Project-specific rules go in the `## Rules (do not break)` section of the area sheet, never in `CLAUDE.md`; leave one pointer line.
- `docs_audit.py score` counts `CLAUDE.md` plus `docs/progress.md` as the always-loaded cost; the `docs-loop` skill shrinks it round by round.
- Placeholders in angle brackets are replaced once; after that the file contains no template wording.

Practical examples

Example 1: a slice instead of a sheet

```
$ grep -n -A 3 '^## Open work' docs/progress/api.md
14:## Open work
15-
16-- [ ] Add pagination to `/search` (new spec item and revision 3).
```

About 120 tokens against about 4,000 for the whole sheet; across a day of sessions the difference is the whole point of the index.

Example 2: moving a rule out of CLAUDE.md

`CLAUDE.md` carried a paragraph about the mobile client's touch targets. The `docs-loop` skill moved it to `## Rules` in `docs/progress/web.md` and left one pointer line. `docs_audit.py score` went from 5,210 to 4,480: every session that does not touch the mobile client stopped paying for it.

Example 3: a second tool, the same rules

A teammate opens the repository in Antigravity. `GEMINI.md` tells it to propose a short branch name before editing and maps the roles to `pro`, `flash` and `flash_lite`; the retrieval commands, the Readiness Gate and the closure checklist are identical, so reviews look the same whichever tool produced the diff.

4. The docs system

TL;DR One index (`docs/progress.md`), one sheet for each area (`docs/progress/<area>.md`) with fixed sections, a roadmap by level, and a list of open decisions. Docs describe the current state, not history; `docs_audit.py` keeps them linked and short.

The index

`docs/progress.md` is the only file `CLAUDE.md` points to. It holds an area table (file, scope, main code), a "which section to open" list, a short list of reference docs and the active milestones. Every other doc must be reachable from a line there; an unlinked area sheet is an **orphan** and costs a penalty.

The area sheet

Copy `docs/progress/core.md` for every area. Three fixed sections, in this order:

- `## How it is today`: one `###` for each module, with the files and the symbols worth searching, and the grep that finds them. State only; no dates, no "we did".
- `## Open work`: checkbox items, each naming the file it touches and the check that proves it done. A closed item is removed, not ticked forever.
- `## Rules (do not break)`: constraints that keep the area working, written as what to do. Added when a constraint appears, deleted when it stops being true.

Size limit: about 120 lines. A longer sheet is split by theme (`<area>-<theme>.md`), linked from its parent and from the index.

The roadmap

`docs/ROADMAP.md` groups items into **levels**; a level starts when the previous one is done. Each row has seven columns:

Column	Meaning
ID	stable identifier (`R1`, `R2`...) used in commits, work orders and area sheets
Pri	1 = do first ... 5 = nice to have
Status	`open`, `active`, `blocked`, `suspended`
Goal	one sentence: what is true when it is done
Requirement	the checkable "done when": a command, a test, a number
Details	where the working notes are
After	the IDs that must finish first, or `-`

Finished items leave the file (git keeps the history). New ideas are appended at the bottom as `R<n>` lines until someone promotes them into a level. Questions that need an owner's decision live in `docs/open-decisions.md`, one section for each area, and are deleted once decided (the result goes into the area sheet).

Practical examples

Example 1: closing a task

Pagination for `/search` is merged. In `docs/progress/api.md` the item leaves `## Open work`; a new rule appears under `## Rules`: "`/search` takes `page` and `page_size`; `page_size` is capped at 100". `docs_audit.py score` still reports no penalty, which is the signal the closure is complete.

Example 2: a roadmap row that says when it is done

```
| ID | Pri | Status | Goal | Requirement | Details | After |
| R7 | 2 | open | Search filters on both clients (M) | gate: 0 blockers for D01 | web | R3 |
```

The requirement is a command anyone can run. "After R3" tells the planner not to start R7 before the release gate work.

Example 3: splitting a long sheet

`docs/progress/api.md` reached 190 lines and `docs_audit.py` added a 200-point penalty. A `docs-loop` round split the auth part into `api-auth.md`, linked it from the index and kept the information (moved, not lost). The score dropped by the penalty and the always-loaded cost did not grow.

5. Subagent roles and routing

TL;DR Seven roles with fixed models, tools and limits: `planner` (plans), `coder` (writes from an order), `mechanic` (zero-risk edits), `light` (judgment, small fixes, verified docs), `deep` (one hard isolated problem), `research` (read-only search), `ops` (already-decided shell commands). The router picks by how specified the task is.

The roles

Agent	Model / effort	Tools	Does	Does not
`planner`	Opus, medium	read-only	Turns vague tasks into Work Orders	write code
`coder`	Haiku, high	read, edit	Writes code from a precise order	design, resolve ambiguity
`mechanic`	Haiku, low	read, edit	Rename, format, templates, progress ticks	make logic decisions
`light`	Sonnet, medium	read, edit	Small fixes, docs verified against code	long or vague jobs
`deep`	Opus, xhigh	read, edit	One isolated critical problem with a reproduction	implement whole features
`research`	Haiku, low	read-only	Search across many files, 5-10 line summaries	modify anything
`ops`	Haiku, low	Bash only	Run decided commands, report PASS/FAIL	decide, fix, kill, merge

An agent file

Each role is a Markdown file in `.claude/agents/` with a front matter and a short body. The front matter is the contract:

```
---
name: coder
description: Writes code EXCLUSIVELY from a precise, exhaustive Work Order. Makes no design decisions.
model: haiku          # opus | sonnet | haiku
effort: high          # low | medium | high | xhigh
tools: Read, Edit, Write, Grep, Glob
---
You are the code-writing arm of <PROJECT>. ...
## Closing states      DONE | BLOCKED: <reason> | FAILED x2
```

The `description` is what the main agent reads when choosing, so it states the trigger and the limit. The body repeats the limits as positive rules and ends with the closing states (section 7). To generate your set: copy the folder, replace `<PROJECT>` and `<test-cmd>`, and change the models if your account offers different ones.

Routing

1. Is the task already specified? Apply the Readiness Gate (section 6). Yes: `mechanic` (pure mechanics), `coder` (code from the order), `light` (needs a little judgment).
2. No: `planner` produces Work Orders first. A vague task never goes straight to an executor, and never to `deep`.
3. An isolated bug with a reproduction that cheap agents failed twice: `deep`, with a declared necessity (section 8).
4. Finding things across many files: `research`. Running commands someone already chose: `ops`.

Antigravity mapping

`AGENTS.md` and `GEMINI.md` map the same roles to Gemini: `planner` and `deep` to `pro`, `coder` and `light` to `flash`, `mechanic` and `ops` to `flash_lite`, `research` to the native research subagent.

Practical examples

Example 1: two bug reports, two routes

"The Save button is 4 px off on mobile": one file, a CSS value, a screenshot test. Gate passes, `light` fixes it. "The index rebuild test sometimes fails": no reproduction, cause unknown. It goes to `planner`, which asks for a loop that triggers it; only once there is a failing command and two failed attempts does `deep` get it.

Example 2: the heartbeat on the cheap agent

An operator's heartbeat is fetch, inbox, ack. Run by the main agent it rereads the whole conversation each time; run by `ops` with a three-line prompt it costs a few thousand tokens. The `ops` file forbids `kill`, `reset --hard` and merges, so delegating shell chores cannot do damage.

Example 3: a summary instead of forty files

"Who reads `page_size`?" goes to `research`, which answers in six lines: `src/server/app.py:14` reads it, the registry `docs/spec/config-keys.toml` lists it, nothing else. The main context gains six lines instead of the file contents.

6. Readiness Gate

TL;DR Five yes/no questions before a task goes to `mechanic`, `coder` or `light`. Five YES: ready. Any NO: the task goes to `planner`, which makes it ready or splits it. The gate is what lets a cheap model work without guessing.

The five questions

#	Question	A NO usually means
1	**Files/functions**: are the files and functions to touch named explicitly?	the agent would have to search and choose
2	**Determinism**: would two people produce the same diff (no open design choice)?	a design decision is hidden in the order
3	**Verifiable criterion**: is there a deterministic command that checks it?	"done" would be an opinion
4	**Limited scope**: about 3 files or fewer?	the task should be split
5	**No blockers**: independent of decisions still pending?	someone has to decide first

The outcome rule

- **5 out of 5 YES**: READY. Choose the executor: `mechanic` if purely mechanical, `coder` if it is code from the order, `light` if a little judgment is needed.
- **Any NO**: NOT READY. `planner` resolves the ambiguity and issues one or more Work Orders, each of which must pass the gate.
- `deep` is not the fallback for a failed gate; it needs a reproduction and a declared necessity.

The Work Order

The skill `work-order` holds the template. The planner fills one block for each step; the executor never has to ask what was meant.

```
### WORK ORDER: WO-12
- **Objective**: reject an empty value in the tag filter.
- **Files/functions to touch**: src/server/search.py:parse_filters(); tests/test_search.py
- **Required change**: when a filter value is empty ("tag="), raise FilterError("empty value").
- **Verification command**: python -m pytest tests/test_search.py -k filters
- **Expected result**: exit 0, 8 passed
- **Assigned agent**: coder
```

Size: a Work Order takes 45 minutes or less (target 15). Larger work is split, one order for each step.

Practical examples

Example 1: a Work Order the gate refuses

```
Objective: improve search relevance.
Files: src/server/search.py (and whatever else is needed)
Change: tune the ranking.
Verify: check that results look better.
```

Q1 NO ("whatever else"), Q2 NO (which tuning?), Q3 NO ("look better" is not a command), Q5 NO (the ranking policy is undecided). Four NOs: it goes to `planner`, which asks the owner to choose the policy and then writes two orders with `pytest -k ranking` as the check.

Example 2: the same task, repaired

WO-12 above names one function and one test file, states the exact behaviour, and gives a command with an expected result. Five YES, assigned to `coder` (Haiku). It finished in 6 tool calls and 60 seconds, status `DONE`.

Example 3: the gate calibrates itself
Rows of the ledger record the gate score. The demo's ledger shows 2 of 7 runs with gate 2 or lower not ending `DONE`, 28.6 percent, and `agent_audit.py` prints "gate too optimistic". The fix was to treat question 2 more strictly: orders that say "port the panel" without naming the files stopped passing.

7. Evidence-based escalation

TL;DR Every agent ends with exactly one status: `DONE` (the verification command passed), `BLOCKED: <reason>` (a decision is missing; back to `planner`), or `FAILED x2` (two attempts failed; move up one level). Claims without evidence are not accepted.

The three states

State	Meaning	What happens next
`DONE`	The verification command of the Work Order ran and passed; a 2-3 line diff summary follows	Integrate; log the run in the ledger
`BLOCKED: <reason>`	Ambiguous specification, mismatched signature or missing decision	Back to `planner`, which answers or splits the order
`FAILED x2`	The verification still fails after two fix attempts	Escalate one level: coder to light, light to deep (with a necessity score)

Why exactly these

- **Evidence over assertion.** DONE is tied to a command, so a reviewer can rerun it. "I think it works" is not a status.
- **BLOCKED is cheap and honest.** The coder agents are told to invent nothing; stopping with the exact blocking point costs far less than a wrong diff.
- **Two attempts, then up.** A third identical attempt rarely succeeds. `FAILED x2` records what was tried so the next level does not repeat it; `deep` rejects a request that lacks this history.
- **The ladder is short.** mechanic and coder escalate to light; light escalates to deep. Anything that needs a decision goes sideways to planner, not up.

The statuses also feed the ledger (section 8): the outcome column accepts `DONE`, `BLOCKED`, `FAILED x2` and `ESCALATED`.

Practical examples

Example 1: coder fails twice, light finishes

WO-2 (a CSV writer) went to `coder`; its test kept failing on quoting. After two attempts it reported `FAILED x2` with the failing assertion. `light` took it, read the real behaviour of the standard library, fixed the quoting and ended `DONE` in 180 s. Ledger rows: `coder ... FAILED x2`, then `light ... DONE`.

Example 2: BLOCKED prevents a guess

WO-4 said "port the filter panel". The coder replied `BLOCKED: decision needed - which filters, and is the date range a picker or two fields?`. `planner` split it into three orders once the owner chose; nothing was built on a guess.

Example 3: DONE without proof is sent back

An agent reported `DONE` for a rename but the build command was never run. The reviewer ran it: two files still used the old name. The rule that DONE needs a passing command is what made the rejection a one-line message instead of an argument.

8. Necessity score, autonomy threshold, ledger and agent_audit

> **TL;DR** Using the expensive model needs a declared score from 1 to 5 with one stated reason. With the autonomy threshold at 4, scores 4-5 switch without asking and 1-3 do not. Every run is logged in `docs/agents/ledger.txt`, and `agent_audit.py` turns the rows into calibration advice.

The score

N	Meaning
1	The cheap model can do it; requested only for convenience
2	Almost certainly the cheap model can do it; the expensive one saves at most one round
3	Likely an error or two rounds with the cheap model: `planner` starts
4	The cheap model already failed, or the error is costly
5	Almost a necessity: the error is irreversible, or the alternative is giving up

A request states **N out of 5, the single reason, and the estimated cost against the cheap model** (for example "about 5x"). There are only two valid reasons: **prior failure** ("a previous model already failed, `FAILED x2`") or **high cost of error** ("resolving or redoing an error is expensive").

The autonomy threshold

`CLAUDE.md` section 3 has one line: **autonomy threshold = 4**. At N=4 or 5 the agent switches to the expensive model without asking, stating score and reason in chat. At N=1-3 it does not. To change how bold the agent is, edit that single number. The score is an estimate; the ledger measures how good the estimates were.

The ledger

After each finished subagent, the main agent appends one line to `docs/agents/ledger.txt`. Append only; never edit old rows.

```
# date|agent|model|task|area|gate|necessity|outcome|tokens|tools|secs
2026-03-05|deep|opus|flaky index rebuild test|web|-|2|DONE|88000|21|420
```

Field	Meaning
date, agent, model	when, which role, which model
task, area	a short description and the area sheet it belongs to
gate	1-5 = how many of the five Readiness Gate answers were YES; `-` if not applicable
necessity	1-5 as declared for the expensive model; `-` for cheap roles
outcome	`DONE`, `BLOCKED`, `FAILED x2`, `ESCALATED`
tokens, tools, secs	integers from the run's report

agent_audit.py

`python scripts/agent_audit.py` prints, for each agent, the outcome shares and the median tokens and seconds, then the %DONE by gate score and by necessity. It warns when runs with a gate of 2 or lower fail more than 20 percent (the gate is too optimistic) and when `deep` ran with a necessity of 3 or lower. Invalid ledger lines go to stderr as `line N: reason` and make the exit code 1.

Practical examples

Example 1: a ledger that shows the expensive model was not needed

```
agent             runs   %DONE  %BLOCKED  %FAILED  %ESCALATED   tok_med  secs_med
coder                4    50.0       25.0     25.0         0.0     17050      57.5
deep                 1   100.0        0.0      0.0         0.0     88000       420
light                2   100.0        0.0      0.0         0.0     33400       145

expensive agent used 1 time(s) with necessity <= 3: the cheap path was enough on paper
```

The `deep` run cost 5 times the median coder run and was logged with necessity 2. At threshold 4 it should not have started without asking; the team now writes the score before the call.

Example 2: moving the threshold with data

After 40 rows, every run with necessity 3 that went to `planner` first ended `DONE` with the cheap model, and the two necessity-4 runs were the only ones the cheap model had failed. The threshold stayed at 4. Had a necessity-3 run failed twice, the line would have moved to 3.

Example 3: catching a bad row

```
$ python scripts/agent_audit.py
line 14: gate must be 1-5 or -
$ echo $?
1
```

A typo in the gate column would silently skew the statistics; the script refuses it and still reports on the valid rows.

9. Consistency gate: spec files, drift, divergences

> **TL;DR** When one fact lives in several files or on several platforms, `consistency.py` compares them. A registry says what is promised; a list of divergences says what differs and why; `check` fails on unexplained drift and `gate` lists what still blocks the release.

The files

File	Holds
`docs/spec/consistency.toml`	which code to compare: `[spec]` (revision constant), `[config]` (how keys are read), `[[match]]` (a truth file and its copies)
`docs/spec/specs.toml`	revisions and items of the specification: id, since, status (`implemented`, `partial`, `planned`), where, platforms
`docs/spec/config-keys.toml`	every config key the code reads: scope, values, default, meaning
`docs/spec/platforms.toml`	the platforms and the capabilities in which they really differ
`docs/spec/divergences.toml`	each known difference: cause (`capability`, `history`, `todo`, `unknown`) and status (`open`, `suspended`, `untested`, `resolved`)
`docs/spec/known.toml`	automatic findings accepted for now, each with a kind and a reason

What `check` verifies

- **match**: for each `[[match]]` block the set of values found by a regular expression in the truth file (say the server's routes) equals the set in every copy (clients, docs).
- **spec**: the revision constant in code equals the registry's last revision; item ids are unique; `where` files exist.
- **config**: every key the code reads is registered with the right scope, and every registered key is still read.
- **divergence**: a difference caused by a capability names one that exists; causes, statuses and evidence files are valid.
- **stale**: an entry in `known.toml` that no rule finds any more must be removed.

An accepted finding's id includes the missing and extra values (`match:routes:src/mobile/routes.js:-/export`), so accepting today's gap does not hide tomorrow's new one.

```
[[match]]
name = "routes"
truth  = { file = "src/server/app.py",    pattern = '@route\("(/[\w-]+)"' }
copies = [
  { file = "src/web/routes.js",    pattern = 'path: "(/[\w-]+)"' },
  { file = "src/mobile/routes.js", pattern = 'path: "(/[\w-]+)"' },
]
```

The principle and the gate

Platforms behave the same **unless a listed capability really differs**. A difference from a capability is accepted; every other difference is a defect, written in `divergences.toml` the day it is found. `consistency.py gate` exits 1 while any `history`, `todo` or `unknown` divergence is `open` or `suspended`, any accepted finding is of such a kind, or any spec item is not `implemented`. `untested` (fixed, not tried on the real platform) does not block but is printed as "to try". `since PATTERN FILE` finds the first release tag containing a pattern, to fill `since` in the registry.

Three numbers are kept apart: the **product version** (every release), the **specification revision** (what clients may rely on; it only grows) and the **container version** (the stored-file layout; it changes only when an old reader must refuse the file).

Practical examples

Example 1: a drift found the day it is made

```
DRIFT   match   routes: src/web/routes.js differs from src/server/app.py
                (missing ['/admin'], extra [])
DRIFT   match   routes: src/mobile/routes.js differs from src/server/app.py
                (missing ['/admin', '/export'], extra [])
consistency: 2 drift, 0 accepted, 1 stale
```

A developer added `/admin` to the server only. The check names the two clients that lack it. The old accepted gap for `/export` is now "stale" because the finding changed shape: it must be re-accepted deliberately, or the clients fixed.

Example 2: a key nobody registered

```
DRIFT   config      the code reads `theme` and docs/spec/config-keys.toml does not list it
```

Fix: add a `[[key]]` block with scope, values, default and meaning. The registry is how support staff and docs learn the key exists.

Example 3: a defect versus a capability

```
$ python scripts/consistency.py gate
BLOCKS RELEASE  D01 (history, priority 2): search filters: port the filter panel to the mobile client
BLOCKS RELEASE  spec item export is partial: web only: the mobile client has no export screen
TO TRY ON THE REAL PLATFORM  D03: haptics: implemented on the phone, not tried on a real device yet
gate: 2 blockers for the release
```

D02 (no export on the phone) is a capability, `no-fs-access`, so it does not block. D01 is only history: nobody ported the panel. That one is a defect and stops the release.

10. Docs audit

TL;DR `docs_audit.py score` gives one number for the health of the docs: tokens that are always loaded, plus the mean area sheet, plus penalties. Lower is better; zero penalties is the goal. The `docs-loop` skill uses it to tidy the docs one measured change at a time.

The score

Tokens are bytes divided by 4. `score = always-loaded tokens (CLAUDE.md + docs/progress.md) + mean tokens of an area sheet + penalties`. The exit code is 1 when any penalty exists.

Penalty	Points	Cause
orphan	500	an area sheet that `docs/progress.md` does not link
broken link	300	a Markdown link, or a backticked path whose first folder exists, pointing at a missing file
oversize	200	an area sheet over about 120 lines
duplicate	50	the same 60-character line in two tracked files
no index pointer	2000	`CLAUDE.md` does not mention `docs/progress.md`

Commands

```
python scripts/docs_audit.py score            # the number and what makes it up (--json for tools)
python scripts/docs_audit.py repeats 30       # prompts/commands repeated in 3+ sessions
python scripts/docs_audit.py log 4480 keep "moved mobile rules to docs/progress/web.md"
python scripts/docs_audit.py best             # last kept score
python scripts/docs_audit.py --root examples/demo score
```

The docs-loop skill

A loop with at most 8 rounds. Each round tests **one hypothesis**, in this order: fix an audit penalty; move area-specific text out of `CLAUDE.md`; split or index a file that is reread a lot; drop an index line that does not help find a file. Copy the touched files to a backup, change, reread the diff (information is moved, not lost), measure, then `log ... keep` or restore and `log ... discard`. It stops after two discards in a row. `repeats` also surfaces workflows worth turning into a skill.

Practical examples

Example 1: a renamed file breaks a link

```
score 1517  (always loaded 550 + mean area 167 + penalties 800)
  ! broken link in docs/progress.md: progress/web.md
  ! orphan: docs/progress/clients.md is not linked from the index
```

`web.md` was renamed to `clients.md` without updating the index: one broken link and one orphan, 800 points, found before anyone reads a stale doc. Fix the link; both penalties vanish.

Example 2: one docs-loop round

Hypothesis: the "design notes" paragraph in `docs/progress/api.md` is duplicated in `CLAUDE.md`. Backup, delete the copy in `CLAUDE.md` and keep a pointer, score 5,210 to 4,980. `log 4980 keep "deduplicate design notes"`. Next round proposes nothing better; the loop stops.

Example 3: a skill candidate

```
repeated commands (3 or more sessions):
    6  git fetch origin && git log
    4  python scripts/consistency.py check
```

The second line is already a script; the first one, with a short summary, becomes a skill or an `ops` prompt. Only repetition that no skill covers is worth writing down.

11. CI audit

TL;DR `ci_audit.py` reads the CI workflow as text and checks the rules that keep it cheap and predictable: a timeout on every job, a concurrency group, sync branches ignored, test waits not shorter than the known wait, required commands present. With `--gh` it adds mean duration and success rate.

The checks

1. Every job has `timeout-minutes`, and the workflow has a `concurrency` block (superseded runs are cancelled; tags are not).
2. `on.push.branches-ignore` lists `sync/**`, so a heartbeat push never starts a run.
3. In the files named by `test_globs`, `expect("text", timeout=T)` is flagged when T is lower than the known wait for that text (`[waits]` in `.ci-audit.toml`).
4. Every command in `required_run` appears in a `run:` step.
5. With `--gh`: `gh run list` gives the mean duration and success share of the last 20 runs (skipped with a warning when `gh` is missing).

```
# .ci-audit.toml
workflow     = ".github/workflows/ci.yml"
test_globs   = ["tests/**/*.py"]
required_run = ["python scripts/selftest.py"]
[waits]
"index rebuild" = 19        # seconds the step really needs
```

The workflow itself is plain text parsed by indentation, so no YAML library is needed. Keep the usual shape: a top-level `concurrency`, a `jobs:` block with two-space children, and `branches-ignore: [reports, 'sync/**']` under `push`. Exit 1 on any finding; run it whenever you touch `.github/workflows/` or a test timeout.

Practical examples

Example 1: a job without a timeout

```
  ! job 'tests' has no timeout-minutes
ci_audit: 1 problem(s) in .github/workflows/ci.yml
```

A hung test job would run until the platform default limit and block the queue. The audit refuses the workflow until the job has a bound sized from real runs.

Example 2: sync pushes that start CI

Operators push heartbeats every few minutes. Without `sync/**` in `branches-ignore`, each one started a full CI run. The check made the omission visible in review, and the fix was one line.

Example 3: a wait that was too short

```
  ! tests/e2e.py:9 waits 5 s for 'index rebuild finished', the known wait is 19 s
```

A test waited 5 seconds for a step that takes 19 and failed one run in three. The table in `.ci-audit.toml` encodes the measured wait, so the flake is caught in the audit, not in the third red build.

12. Multi-operator sync

> **TL;DR** Several agents (Claude local or cloud, Antigravity, others) work in parallel through a written protocol: a supervisor (SPV) plans and reviews, operators deliver on their own branches, each file has one writer, inboxes carry tasks, one delivery line and one review line close each work order, and a script reports who is alive. The protocol lives only on a branch `sync/<name>`.

Roles, files and the sync branch

SPV writes `SYNC.md` (contracts, work orders, decisions, the Log), `sync/wo/*` and `sync/inbox/*`. Every other operator writes only its own `sync/<ID>.md`: state, tasks, deliveries, progress, problems, requests. Everyone reads everything. Operators are listed in `sync/operators.toml`. The **user** decides and does the final merges.

Each project has one sync branch `sync/<name>` cut from the main branch. `SYNC.md`, `sync/*` and `tools/sync_*.py` are used and pushed only there (`git pull --rebase`, never a merge commit); the main branch keeps the templates, so two sync sessions never touch each other. **Code** lives on working branches (`claude/<topic>`) and reaches the main branch only after `REVIEW: APPROVED` and with the user's consent. SPV writes from a separate worktree so it never switches its own working branch.

States and the status block

State	Meaning
`WORKING {task}`	working on that task
`WAITING {who}`	blocked until the user, another operator or a service acts
`FREE`	agreed tasks finished and pushed; available
`DONE`	terminal: session over
`IDLE`	WORKING/WAITING more than 10 minutes past `eta` with no update (no `eta`: 60 minutes); never applied to FREE or DONE

The block at the top of `sync/<ID>.md` is written by `tools/sync_hdr.py`, never by hand: `schema`, `id`, `state`, `task`, `waiting_on`, `progress`, `eta`, `wakeup`, `ack`, `updated`. Times are ISO 8601 with a zone. `eta` is mandatory in WORKING and WAITING; `ack` is the read receipt (the last Log row read).

```
python tools/sync_hdr.py ALPHA --state WORKING --task "WO-12" --eta 2026-10-01T11:30+02:00 --wakeup no
python tools/sync_hdr.py ALPHA --ack auto          # heartbeat: updated + ack, state and task untouched
```

Handshake, assignment, inbox

1. The user says "do the handshake `sync/<name>`". The operator fetches, checks the branch out in a separate worktree, pulls with rebase.
2. It reads `SYNC.md` (roles, contracts, work orders, rules) and `sync/operators.toml`.
3. It creates only `sync/<ID>.md` from `sync/_template.md`: the status block, `## Profile` (where it runs and breaks, models, subagents, tools) and `## Requirements` if section 2 lists them.
4. Commit, `git pull --rebase`, push to `sync/<name>`, then wait. **No work starts before SPV writes the assignment.**

Every task and urgency arrives in `sync/inbox/<ID>.md`: at most 5 lines, one task, with work order, branch, verify command and `eta`. The heartbeat reads the `git diff` of that file, then `--ack auto`.

Delivery, review, sign-off

A finished work order is one line in `## Deliveries`, then `FREE`:

```
DELIVERED WO-1b @ claude/search-filters@e4f5a6b · verify: python -m pytest tests/test_search.py -k filters
    -> 8 passed · 2026-10-01T11:10+02:00        (one line in the file; wrapped here)
```

`DONE` is not a delivery; a non-standard signal starts nothing. SPV does not trust the report: an **independent** subagent reviews the delivered commit in a temporary worktree: (1) perimeter against the files the work order names, (2) rerun every verify command and classify failures as pre-existing (same error on a clean base) or introduced, (3) read the diff

for logic and security defects, (4) per-work-order verdict with `file:line`. SPV then writes one Log row, `REVIEW: APPROVED` or `REVIEW: CHANGES REQUESTED`, of at most 600 characters. Only after APPROVED does integration happen, in SPV's integration branch.

Sign-off: when SPV writes `SIGNOFF <ID>: reason`, the operator starts nothing new, delivers and pushes what exists, writes its closing line, marks itself `DONE` and stops its heartbeats. It returns only with a new handshake. A sign-off is not a fault; delivered work stays valid.

Heartbeat and checks

The heartbeat is event-driven, with no timer: update your file and push at every finished task; a long task is split with a progress line; a problem or a request is a pushed row (`WAITING`) plus one sentence in chat. The cost depends on who runs it:

Who runs the beat	Cost of one beat
The main agent (a timer re-sending a prompt)	rereads the whole conversation: the most expensive
A small subagent (cheap model, empty context)	about 5-15 thousand tokens
`tools/sync_watch.py` (no model)	0 tokens; wakes the agent only when its inbox or an addressed row changes

`python tools/sync_check.py [--strict]` is the objective view SPV reports from: age of each block, `eta`, `ack`, unread rows addressed to an operator, deliveries out of format, inboxes over 5 lines, Log rows over 600 characters, and IDLE operators. Fixed rules: one writer for each file, tasks of 45 minutes or less, the main agent stays free, messages between operators are a block `from <ID> to <ID>: ...` in chat, hypotheses are labelled `HYPOTHESIS`, and a rule change is a `RULE` row valid for an operator only after its `ack`.

Practical examples

Example 1: CHANGES REQUESTED, with evidence

```
2026-10-01T10:40+02:00 | ALPHA | claude/search-filters@a1b2c3d | REVIEW: CHANGES REQUESTED |
WO-1: perimeter ok, 6 tests pass, but parse_filters("tag=") returns every task. Reviewer
reproduced it with a new test (-k filters_empty, fails). Fix in WO-1b, in your inbox.
Nothing outside src/server/search.py and its test.
```

The row names the commit, the reproduction and the next step. ALPHA read it, wrote `read 10:40` in its progress, delivered WO-1b, and the new test failed on the old code (0/1) and passed on the fix, so the approval was easy.

Example 2: a freeze between two operators avoided by the perimeter

ALPHA owns `src/server/search.py`, BETA owns `src/mobile/*`; the shared routes file is named in one Work Order, "one line each". When both needed `src/web/routes.js` on the same day, the order of the lines was decided by SPV in the inbox instead of by whichever push landed last. No conflict, no overwritten line, and the review diff checked the perimeter in seconds.

Example 3: a silent operator found by the check

```
  ! BETA: IDLE by rule (no update for 95 min, eta 2026-10-01T11:25+02:00); tell the user
1 problem(s)
```

BETA's container had slept and its background job was gone. `sync_check` flagged it past `eta` plus 10 minutes. SPV told the user in chat; a message in BETA's session was the only channel that works for a silent operator, and BETA resumed within minutes.

13. Writing rules as positive instructions

TL;DR State each rule as what to do, not as a prohibition. A "do not X" line keeps the unwanted behaviour in view and reads like a new rule when you delete it. Retire a rule by deleting it. Add an explicit prohibition only as an exception, with the harm written next to it.

The rule

`CLAUDE.md` section 2 carries the instruction itself: state each rule as what to do ("do it this way"); when a rule is retired, delete it and do not replace it with its negation; add an explicit prohibition only when the agent keeps repeating something that harms development despite the positive rule, and note the harm next to it.

Why it works

- A prohibition names the very thing you want out of the model's attention. Naming it often makes it more likely.
- A positive rule is a procedure the agent can follow; a negative rule only leaves a gap that something else fills.
- Deleting a retired rule leaves no residue. Replacing it with "do not do the old thing" turns history into an instruction.
- A prohibition with a written harm is still checkable: you know when it can be dropped (when the harm stops happening).

Rewriting patterns

Before	After
Do not edit generated files by hand.	Change the generator input, then run `<build-cmd>`; commit both.
Never read entire log files.	Search the log with `grep -n <pattern>`, then read the span around the hit.
Do not guess when a requirement is unclear.	Stop with `BLOCKED: <the exact question>`.
Do not commit secrets.	Keep keys in the environment or the secret store; the repository holds only their names.
Don't use the expensive model for simple work.	Use the expensive model at necessity 4 or 5; below that start with `planner`.

The exception

When the agent keeps doing something harmful despite a positive rule, add one explicit prohibition and write the harm beside it, so the line can be removed when the harm no longer occurs:

Example: "Run migrations through `<build-cmd> migrate`. Exception, added 2026-05: never run raw SQL against the shared database; it dropped the staging data twice."

Practical examples

Example 1: a rule rewritten

Before: "Do not use global state in handlers." After: "Pass request state through function arguments." The agent stopped introducing module-level caches because the positive rule told it what to build instead.

Example 2: retiring a rule cleanly

The rule "run the legacy importer before tests" stopped being true when the importer was removed. The line was deleted. A teammate suggested adding "do not run the importer"; that would have taught every future session that an importer exists, so it was not added.

Example 3: an exception with its harm

An agent kept force-pushing sync branches to "fix" a rejected push. The positive rule (`git pull --rebase origin sync/<name>`) was already there, so one exception was added: "never force-push a sync branch: it erased two operators' deliveries on 2026-10-01". When a month passes without a repeat, the line goes.

14. Bootstrapping step by step

TL;DR From zero to a working setup in about an hour: copy the kit, replace placeholders, write the index and the first area sheets, register what you keep consistent, put the self-test in CI, and push one real task through the planner and the gate. The sync protocol is added only when a second operator joins.

Checklist with commands

1. **Copy the kit** into the repository: `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`, `.claude/`, `docs/`, `scripts/`, `tools/`, `SYNC.md`, `sync/`, `.github/workflows/ci.yml`, `.ci-audit.toml`. Leave `examples/` out if you do not want the demo, but keep `scripts/selftest.py` and edit its demo paths.
2. **Replace placeholders**: `grep -rn '<PROJECT>\|<main-branch>\|<build-cmd>\|<test-cmd>' CLAUDE.md AGENTS.md GEMINI.md .claude` and fix each hit.
3. **Write the index and the area sheets**: `docs/progress.md` with one row for each area; copy `docs/progress/core.md` for each area; fill `How it is today`, `Open work`, `Rules`.
4. **Add the roadmap**: levels in `docs/ROADMAP.md` with ID, Pri, Status, Goal, Requirement, Details, After; one row is enough to begin.
5. **Register what you keep in step**: in `docs/spec/consistency.toml` add a `[[match]]` for each fact that lives in several files, a `[config]` block if you have config keys, and a `[spec]` revision constant; then fill `specs.toml`, `config-keys.toml`, `platforms.toml`.
6. **Fix the CI**: give every job `timeout-minutes`, add `concurrency`, `branches-ignore: [reports, 'sync/**']`; put your test command in `required_run` of `.ci-audit.toml`.
7. **Prove it**: run the checks below until each exits 0.
8. **Push one real task through**: describe it vaguely to `planner`, get Work Orders, check each with the gate, run `coder` or `light`, close with the checklist, append a ledger row.
9. **Add sync when a second operator joins**: create `sync/<name>`, copy `SYNC.md`, `sync/`, `tools/sync_*.py` there, fill sections 2 and 3, then say "do the handshake `sync/<name>`" in each operator's session.

```
python scripts/selftest.py                  # the kit itself
python scripts/docs_audit.py score          # 0 penalties
python scripts/consistency.py check         # 0 drift
python scripts/consistency.py gate          # list what blocks your first release
python scripts/ci_audit.py                  # 0 problems
git switch -c sync/<name>                   # only when a second operator joins
```

What to customise first

- The invariants in `CLAUDE.md` section 4 and the first two rules of every area sheet: they are what an agent cannot infer from code.
- The models in `.claude/agents/*.md` if your account names them differently.
- The autonomy threshold (section 3 of `CLAUDE.md`): start at 4, adjust from the ledger after about 30 runs.

Practical examples

Example 1: a bootstrap timeline for Taskly

Minute 0-10: copy the kit, replace placeholders. 10-25: `docs/progress.md`, `api.md` and `web.md` written from the existing code. 25-40: two `[[match]]` blocks (routes, error codes), the revision constant, four config keys registered. 40-50: CI fixed (two timeouts added, `sync/**` ignored). 50-60: `selftest.py` green, first Work Order through the gate. The score was 717 with no penalty.

Example 2: the first real task

"Add pagination to search." `planner` returned two orders; the first (server parameter, files and test named) passed the gate and went to `coder` (`DONE`, 60 s); the second (client UI) failed Q5 because the page size was undecided, so it became an item in `docs/open-decisions.md`. The closure checklist then updated `Open work` and added one rule.

Example 3: adding an area later

```
cp docs/progress/core.md docs/progress/billing.md
# edit it, then add one row to the area table of docs/progress.md
python scripts/docs_audit.py score
```

If the row is forgotten, the new sheet is an orphan and the score jumps by 500, which is how the omission is found.

15. Maintenance and metrics

TL;DR The workflow stays useful only if it is measured and pruned. A short routine at each task, milestone and month; a few numbers to watch; and the habit of deleting rules and files that stopped being true.

The routine

When	What
Every task	Closure checklist (`CLAUDE.md` section 5): update `Open work` and `Rules`, run `docs_audit.py score`; `consistency.py check` and `ci_audit.py` when relevant; one ledger row for each subagent run
Every milestone	`docs-loop` rounds (at most 8); update the ROADMAP row; read `consistency.py gate`; resolve or re-date `untested` divergences
Every month	Read `agent_audit.py`: adjust the autonomy threshold and the Readiness Gate strictness; prune rules that are no longer true; run `ci_audit.py --gh`
Whenever a script changes	`python scripts/selftest.py`, and add a negative check for the new behaviour
Whenever the workflow changes	Update `docs/WORKFLOW.md` and rebuild this PDF with `python scripts/build_workflow_pdf.py`

Metrics to watch

Metric	Source	Healthy
Docs score and penalties	`docs_audit.py score`	0 penalties; always-loaded tokens flat or falling
Consistency drift and gate blockers	`consistency.py check`, `gate`	0 drift; blockers falling toward the release
%DONE by gate score	`agent_audit.py`	gate 1 near 100 percent; no "gate too optimistic" warning
Expensive-model use at low necessity	`agent_audit.py`	no warning; `deep` rows have necessity 4 or 5
CI mean duration and success rate	`ci_audit.py --gh`	stable minutes; no run longer than its timeout
IDLE operators and unread rows	`sync_check.py`	0 problems when a sync is active

Pruning

- Delete a rule when it stops being true; do not write its negation (section 13).
- Remove a closed item from `Open work`; remove a decided question from `open-decisions.md`.
- Remove `known.toml` entries that `check` calls stale; resolve or reclassify old `suspended` divergences.
- Archive a finished `SYNC.md` in `docs/archive/` on the main branch and close the sync branch.

Practical examples

Example 1: a monthly ledger review

Forty rows later `agent_audit.py` showed 96 percent `DONE` at gate 1, 71 percent at gate 2, and no necessity 3 run that needed the expensive model. Decision: keep the threshold at 4, tighten question 2 of the gate for orders that mention "port" without file names, and note both in `docs/open-decisions.md`.

Example 2: a score trend

```
$ python scripts/docs_audit.py best
4480
$ python scripts/docs_audit.py score
score 4310  (always loaded 3105 + mean area 505 + penalties 700)
```

The best kept score is 4,480 and now there are 700 points of penalties: two oversized sheets. The next `docs-loop` round splits them, and the score returns to the trend line.

Example 3: retiring a skill

The skill `sd-card-flash` described a manual step a CI job now does. `docs_audit.py repeats` showed zero sessions using it in 60 days. The skill folder was deleted, the one-line mention in the area sheet removed, and `selftest.py` still passed, which proved nothing else depended on it.